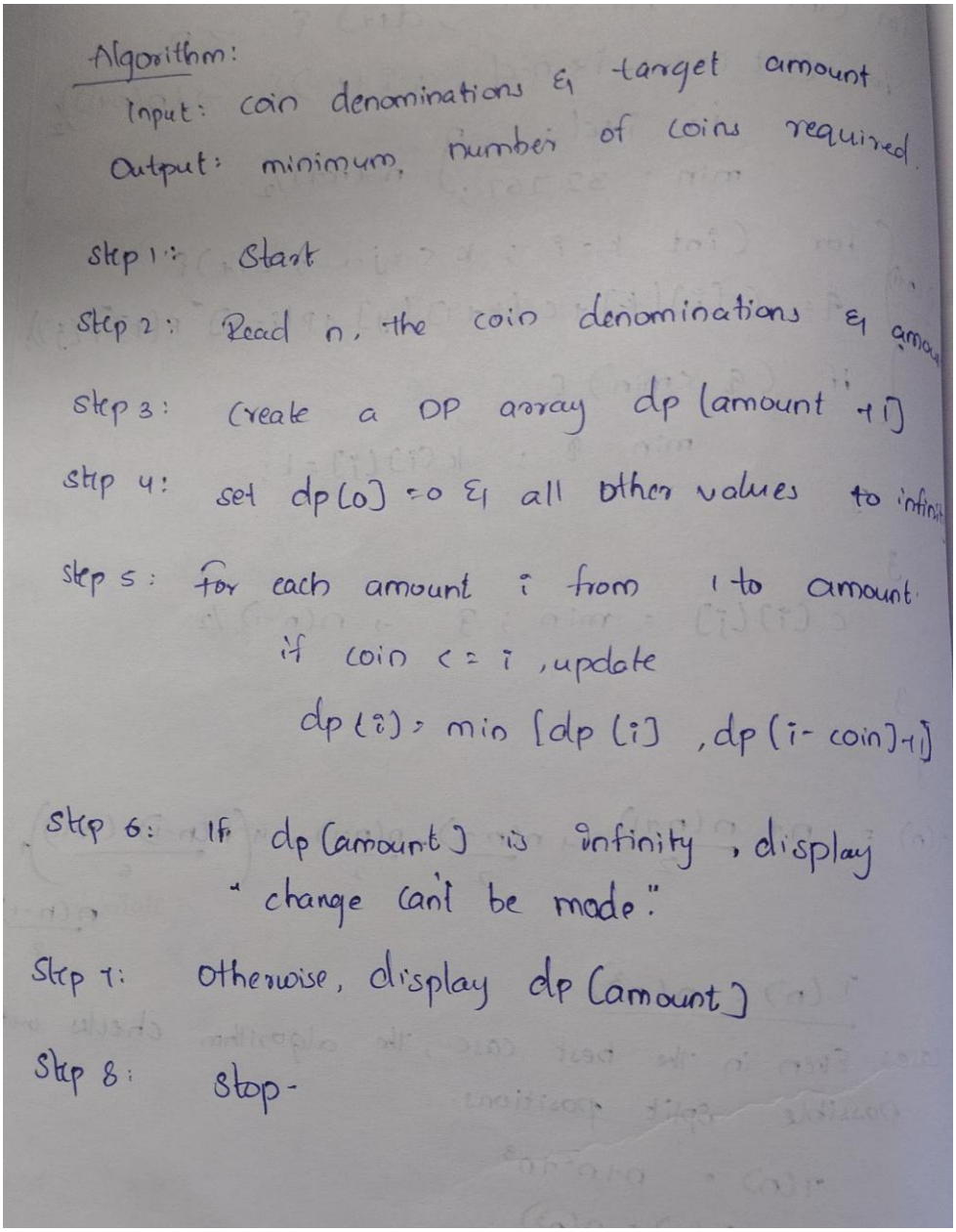
PRACTICAL: 07

Aim: Implementation of making a change problem using dynamic programming.

Algorithm:



Algorithm:
Input: coin denominations & target amount
Output: minimum number of coins required

Step 1: Start
Step 2: Read n, the coin denominations & amou
Step 3: Create a DP array dp[amount + 1]
Step 4: set dp[0] = 0 & all other values to infini
Step 5: for each amount i from 1 to amount
if coin <= i, update
dp[i] = min [dp[i], dp[i - coin] + 1]
Step 6: If dp[amount] is infinity, display "change can't be made."
Step 7: Otherwise, display dp[amount]
Step 8: Stop.

Program / Code:

```
#include  <iostream>
using namespace std;

class ChangeMaking
{
private:
    int coins[10];
    int n;
    int w;
    int A[10][100];

public:

    void getInput()
    {
        cout << "Enter number of coin types: ";
        cin >> n;

        cout << "Enter coin values: ";
        for (int i = 1; i <= n; i++)
        {
            cin >> coins[i];
        }

        cout << "Enter amount: ";
        cin >> w;
    }
```

```
void solve()
{
   // There is 1 way to make amount 0:
   // select no coins
   for (int i = 0; i <= n; i++)
   {
      A[i][0] = 1;
   }

   // With 0 coin types, no positive amount can be made
   for (int j = 1; j <= w; j++)
   {
      A[0][j] = 0;
   }

   // Fill the DP table
   for (int i = 1; i <= n; i++)
   {
      for (int j = 1; j <= w; j++)
      {
         if (coins[i] > j)
         {
            A[i][j] = A[i - 1][j];
         }
         else
         {
            A[i][j] = A[i - 1][j]
```

```
                + A[i][j - coins[i]];
            }
        }
    }
}

void printTable()
{
    cout << "\nDP Table:\n";

    cout << "Coin\t";

    for (int j = 0; j <= w; j++)
    {
        cout << j << "\t";
    }

    cout << endl;

    for (int i = 0; i <= n; i++)
    {
        if (i == 0)
            cout << "0\t";
        else
            cout << coins[i] << "\t";

        for (int j = 0; j <= w; j++)
        {
```

```
            cout << A[i][j] << "\t";
        }

        cout << endl;
    }
}

void showResult()
{
    cout << "\nTotal Number of Ways = "
         << A[n][w] << endl;
}
};

int main()
{
    ChangeMaking c;

    c.getInput();
    c.solve();
    c.printTable();
    c.showResult();

    return 0;
}
```

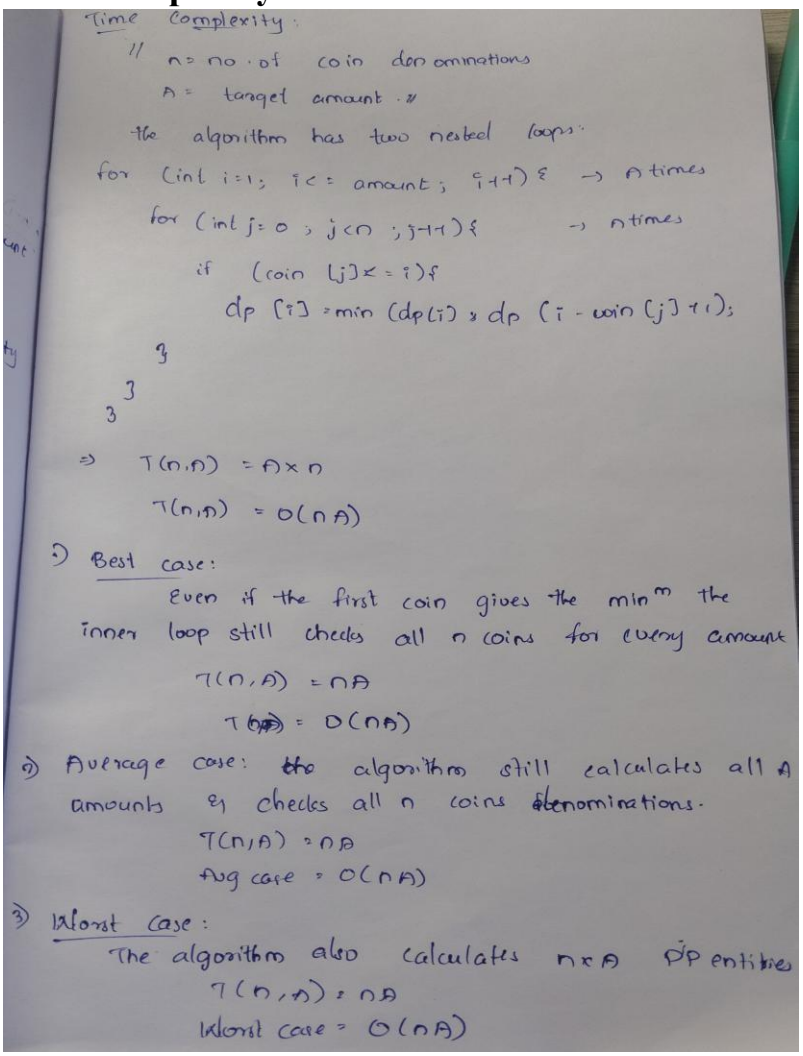
Output:

```
PS C:\Users\pusap\Downloads\c++> C:\Users\pusap\Downloads\c++\makingchangeproblem.exe
Enter number of coin types: 4
Enter coin values: 2
4
6
3
Enter amount: 8

DP Table:
Coin    0       1       2       3       4       5       6       7       8
0       1       0       0       0       0       0       0       0       0
2       1       0       1       0       1       0       1       0       1
4       1       0       1       0       2       0       2       0       3
6       1       0       1       0       2       0       3       0       4
3       1       0       1       1       2       1       4       2       5

Total Number of Ways = 5
PS C:\Users\pusap\Downloads\c++>
```

Time Complexity:



Conclusion:

The Dynamic Programming method efficiently calculates the total number of ways to make a given amount using the available coins. It avoids repeated calculations by storing previously computed results.